

Healthcare Appointment Capacity & Queue Management Platform

Complete Project Plan

Deadline: 30 September 2026

Technology: Java + Spring Boot + JDBC + Hibernate/JPA + MySQL + React

Scope decision: No AI/ML for the current deadline. The project focuses on rule-based optimisation, capacity management, queue management, analytics, security and auditability.

Project positioning: A healthcare operations platform that dynamically manages appointment capacity and patient queues while reducing scheduling conflicts, waiting time and unused doctor capacity.

Source basis: The original brief covers patient, doctor, department, availability, appointment, queue, priority, cancellation/rescheduling, history, capacity optimisation, dashboard and audit modules. ■filecite■turn0file0■L17-L39■

1. Core Objective

The system should do more than store appointments. It should calculate available capacity, recommend suitable appointment slots, organise queues using configured rules, prevent double booking and provide managers with operational information such as utilisation, waiting time, cancellations and department workload. The brief explicitly requires capacity calculation, conflict prevention and operational analysis. ■filecite■turn0file0■L11-L16■

2. Final Feature Set

Module	Final scope
Authentication	Login, password hashing, role-based access
Patient Management	Registration, profile, appointment/visit history
Doctor Management	Profile, specialisation, department, consultation status
Department Management	Departments and operating schedules
Availability	Working days, consultation hours, breaks and leave
Appointment Management	Book, cancel, reschedule, status tracking
Smart Slot Recommendation	Rule-based recommendation using availability, duration and existing bookings
Queue Management	Token, queue position, status and configured priority
Waiting-Time Estimate	Estimated waiting time based on queue and consultation duration
Priority Management	Configurable priority categories and transparent queue rules
Capacity Optimisation	Calculate capacity and identify suitable appointment opportunities
Doctor Workload	Utilisation and workload monitoring
Cancellation Recovery	Released slots become available for eligible future bookings
Operational Dashboard	Appointments, queues, utilisation, cancellations and workload
What-if Capacity Simulator	Optional final differentiator if core modules are stable
Audit Management	User, action and timestamp history for changes
Reports	Operational summaries; PDF export only if time permits

3. What Makes the Project Stand Out

A. Smart appointment slot recommendation: Instead of simply listing every free slot, the backend evaluates doctor availability, appointment duration, breaks, leave and existing bookings and returns suitable options.

B. Dynamic queue and waiting-time estimation: The system displays the patient's token, patients ahead and an estimated waiting time. The brief already requires queues to respond to completed visits, cancellations and priority rules.

■filecite■turn0file0■L34-L39■

C. Doctor workload/utilisation: Managers can see which doctors have high or low utilisation and monitor department workload.

D. Cancellation slot recovery: A cancelled appointment releases capacity and the slot can be considered for future bookings, extending the brief's cancellation requirement. ■filecite■turn0file0■L41-L43■

E. Explainable priority: The UI should show the configured priority category/rule behind a queue position rather than presenting an unexplained number.

F. What-if capacity simulation: If time permits, managers can simulate changes such as doctor leave or consultation duration and see the resulting capacity and affected appointments.

4. System Architecture

Frontend: React → REST API → **Backend:** Spring Boot → Controller → Service → Repository → Hibernate/JPA →

Database: MySQL.

Use JDBC selectively for direct SQL and specialised analytics/reporting queries. Use Hibernate/JPA for entity mapping and normal CRUD operations. This gives you a clear technical explanation for why both technologies are present.

Suggested packages: controller, service, repository, entity, dto, exception, security, scheduler, analytics, util.

5. Main Database Design

Table	Purpose
users	Login accounts and user identity
roles	ADMIN, RECEPTIONIST, DOCTOR, MANAGER
patients	Patient demographic/contact information
departments	Hospital departments and operating information
doctors	Doctor profile, specialisation and department
appointment_types	Appointment duration/type
doctor_availability	Working days, consultation hours and breaks
doctor_leave	Doctor leave periods
appointments	Booking, date/time, status and relationships
appointment_history	Completed, cancelled, missed and rescheduled history
queue_entries	Token, position, status and priority
priority_rules	Configured queue priority categories/rules
audit_logs	User, action, timestamp and changed record
notifications	Optional reminder/notification records

6. Key Relationships

Department 1:N Doctor; Doctor 1:N Availability; Doctor 1:N Appointments; Patient 1:N Appointments; Appointment 1:1 Queue Entry; Appointment 1:N History Records; User 1:N Audit Logs.

7. Core Business Rules

- Prevent double booking for the same doctor/date/time.
- Do not allow an appointment outside the doctor's working hours.
- Do not allow appointments during configured breaks.

- Doctor leave blocks appointment creation for the affected period.
- Appointment duration must fit completely within the available slot.
- Cancellation releases the appointment capacity.
- Rescheduling must validate the new slot using the same rules as a new booking.
- Queue ordering follows configured priority and operational rules; unauthorised manual manipulation is prevented.
- Every important appointment/queue modification creates an audit record.
- Waiting-time estimates should be labelled as estimates, not guaranteed times.

8. REST API Plan

Area	Example endpoints
Auth	POST /api/auth/login
Patients	POST/GET /api/patients; GET/PUT /api/patients/{id}
Doctors	POST/GET /api/doctors; GET/PUT /api/doctors/{id}
Availability	POST /api/availability; GET /api/availability/doctor/{doctorId}
Appointments	POST/GET /api/appointments; GET/PUT /api/appointments/{id}
Smart Slots	GET /api/appointments/recommended-slots
Queue	GET /api/queue/today; GET /api/queue/{doctorId}; PUT /api/queue/{id}/status
Dashboard	GET /api/dashboard/summary
Analytics	GET /api/analytics/utilisation; GET /api/analytics/workload
Audit	GET /api/audit-logs

9. React Frontend

Recommended pages: Login, Dashboard, Patients, Doctors, Departments, Availability, Appointments, Queue, Reports, Audit Logs and Role-specific views.

Role-specific dashboards: Admin manages configuration/users; Receptionist handles patients and appointments; Doctor sees their appointments and queue; Manager sees capacity, utilisation, waiting time, cancellations and workload.

10. Recommended Demonstration Flow

Patient request → select department/doctor → check availability → check leave/breaks → check existing bookings → calculate capacity → recommend slots → book appointment → generate queue position → estimate waiting time → update dashboard → write audit history.

11. Priority for the Deadline

Priority	Work
Must have	Login, patient, doctor, department, availability, appointments, cancellation, rescheduling, queue, priority, history, dashboard, My
Strong differentiators	Smart slot recommendation, waiting-time estimate, doctor utilisation, cancellation slot recovery, role-based dashboard and audit
Only if core is stable	What-if simulator, notifications, attendance/no-show indicator, advanced reports and PDF export.
Do not add now	AI/ML, chatbot, payment gateway, real SMS integration, EMR integration, blockchain, mobile app or complex microservices.

12. Final Project Pitch

"Our project is a Healthcare Appointment Capacity and Queue Management Platform developed using Java, Spring Boot, Hibernate, JDBC, MySQL and React. The objective is not just to book appointments, but to optimise available capacity and manage patient queues. The system considers doctor availability, appointment duration, existing bookings, breaks and leave periods before recommending suitable appointment slots. It also calculates queue position and estimated waiting

time, monitors doctor utilisation and department workload, and maintains appointment history and audit records. Role-based access ensures that administrators, receptionists, doctors and managers have appropriate responsibilities."

Standout line: "The key differentiator is the capacity and queue optimisation engine, which converts appointment and availability data into scheduling and operational recommendations instead of simply storing bookings."